ЗВІТ з лабораторної роботи №6

Тема: Розробка консольного мережевого сніфера з використанням WinAPI та сокетів

1. Мета роботи

Ознайомитись із можливостями WinAPI та Winsock2 для роботи з мережевими інтерфейсами, навчитися використовувати низькорівневі (raw) сокети в режимі прийому всіх пакетів (SIO_RCVALL), отримувати мережеві пакети в реальному часі та вручну розбирати заголовки протоколів IPv4, TCP, UDP та ICMP.

2. Теоретичні відомості

2.1. Мережевий сніфер

Мережевий сніфер — це програма, яка перехоплює та аналізує мережеві пакети, що проходять через мережевий інтерфейс. Сніфери використовують для діагностики й моніторингу мережі та вивчення протоколів.

2.2. Raw-сокети та режим SIO_RCVALL

Raw-сокет дозволяє працювати безпосередньо з IP-пакетами разом із заголовками протоколів. Він створюється викликом socket(AF_INET, SOCK_RAW, IPPROTO_IP). Після прив’язки до локальної IP-адреси (bind) сокет переводиться в режим прийому всіх пакетів викликом ioctlsocket(sock, SIO_RCVALL, &flag), де flag = RCVALL_ON (1). Для цього потрібні права адміністратора.

2.3. Важлива особливість Windows

У методичних матеріалах зазначено, що raw-сокет повертає Ethernet-кадр. Насправді у Windows сокет із SIO_RCVALL повертає пакет, який починається одразу з IP-заголовка: Ethernet-заголовка (MAC-адрес призначення й джерела, EtherType) у даних немає. Тому в програмі розділ [ETH] виводиться зі значенням N/A («недоступно через raw-сокет»), а структура eth_header оголошена згідно із завданням, але для розбору не використовується. Тип 0x0800 (IPv4) визначається за полем Version в IP-заголовку. Щоб отримати справжні Ethernet-кадри, потрібні

драйвер захоплення (наприклад, Npcap), але це виходить за межі завдання.

2.4. Формати заголовків

Ethernet II (14 байт; у raw-сокеті Windows відсутній):

Поле	Розмір	Опис
Destination MAC	6 байт	MAC-адреса призначення
Source MAC	6 байт	MAC-адреса джерела
EtherType	2 байти	0x0800 — IPv4, 0x0806 — ARP, 0x86DD — IPv6

IPv4-заголовок (20–60 байт):

Поле	Розмір	Опис
Version + IHL	4 + 4 біти	Версія (4) і довжина заголовка в 32-бітних словах
Type of Service	1 байт	DSCP / ECN
Total Length	2 байти	Довжина всього пакета
Identification	2 байти	Ідентифікатор для фрагментації
Flags + Fragment Offset	3 + 13 біт	DF, MF і зміщення фрагмента
TTL	1 байт	Час життя пакета
Protocol	1 байт	1 — ICMP, 6 — TCP, 17 — UDP
Header Checksum	2 байти	Контрольна сума заголовка
Source IP	4 байти	IP-адреса джерела
Destination IP	4 байти	IP-адреса призначення

TCP-заголовок (мінімум 20 байт):

Поле	Розмір	Опис
Source Port	2 байти	Порт джерела
Destination Port	2 байти	Порт призначення
Sequence Number	4 байти	Порядковий номер
Acknowledgment	4 байти	Номер підтвердження

Поле	Розмір	Опис
Number		
Data Offset + Reserved + Flags	4 + 3 + 9 біт	Довжина заголовка та прапорці (NS, CWR, ECE, URG, ACK, PSH, RST, SYN, FIN)
Window Size	2 байти	Розмір вікна
Checksum	2 байти	Контрольна сума
Urgent Pointer	2 байти	Вказівник термінових даних

UDP-заголовок (8 байт):

Поле	Розмір	Опис
Source Port	2 байти	Порт джерела
Destination Port	2 байти	Порт призначення
Length	2 байти	Довжина заголовка й даних
Checksum	2 байти	Контрольна сума

2.5. Порядок байтів

У мережі багатобайтові числа передаються у форматі big-endian (network byte order), а процесори x86/x64 використовують little-endian. Тому всі 16- і 32-бітні поля (порти, довжини, seq, ack тощо) перетворюються функціями ntohs() та ntohl().

3. Алгоритм роботи програми

1. Ініціалізація Winsock: WSAStartup(MAKEWORD(2,2), &wsa) з перевіркою результату.
2. Отримання списку локальних IPv4-адрес (gethostname + getaddrinfo) і вибір інтерфейсу користувачем.
3. Створення сирого сокета: socket(AF_INET, SOCK_RAW, IPPROTO_IP).
4. Прив’язка сокета до обраної IP-адреси викликом bind(); встановлення таймауту recv (1 с), щоб програму можна було зупинити по Ctrl+C.
5. Увімкнення режиму прийому всіх пакетів: ioctlsocket(sock, SIO_RCVALL, &flag) з flag = RCVALL_ON.

6. Нескінченний цикл: recv() у буфер 65536 байт, перевірка довжини, розбір IP-заголовка (версія, IHL, TTL, Total Length, Protocol, адреси, прапорці фрагментації).
7. Залежно від поля Protocol розбирається TCP (порти, прапорці, seq, ack, window), UDP (порти, довжина) або ICMP (тип, код).
8. Виведення інформації про кожен пакет у консоль у форматі [ETH] / [IP] / [TCP|UDP|ICMP].
9. Завершення: вимкнення SIO_RCVALL (RCVALL_OFF), closesocket(), WSACleanup().

4. Інструкція із запуску

4.1. Збірка

1. Встановіть Visual Studio з компонентом «Розробка класичних застосунків на C++» (Desktop development with C++).
2. Покладіть файл sniffer.cpp у будь-яку папку (наприклад, C:\lab6).
3. Відкрийте «Developer Command Prompt for VS» від імені адміністратора (ПКМ → Run as administrator).
4. Перейдіть до папки та зберіть програму командою: cl /EHsc /utf-8 sniffer.cpp

Бібліотека ws2_32.lib підключається автоматично директивою #pragma comment. Ключ /utf-8 потрібен для коректного відображення українського тексту.

4.2. Збірка через Visual Studio (альтернатива)

- Створіть проєкт «Console App» (C++), замініть вміст головного файлу кодом із sniffer.cpp.
- Project → Properties → C/C++ → Command Line → Additional Options: додайте /utf-8.
- Запускайте Visual Studio від імені адміністратора й виберіть Debug → Start Without Debugging (Ctrl+F5).

4.3. Запуск і перевірка

1. Запустіть sniffer.exe від імені адміністратора (у консолі, відкритій як адміністратор).
2. Оберіть номер інтерфейсу зі списку (IP вашого Wi-Fi або Ethernet-адаптера) і вкажіть кількість пакетів (0 — без обмеження).

3. Згенеруйте трафік: відкрийте сайт у браузері, виконайте у другому вікні ping 8.8.8.8 та nslookup google.com.
4. Зупиніть програму натисканням Ctrl+C.

4.4. Можливі проблеми

- Помилка 10013 (WSAEACCES) — програму запущено без прав адміністратора.
- Пакетів немає — обрано не той інтерфейс (наприклад, віртуальний адаптер); спробуйте іншу IP-адресу з вашого поточного підключення.
- Не видно частини вихідного трафіку або чужих пакетів — це обмеження Windows і драйвера адаптера (особливо для Wi-Fi); порівняйте з Wireshark.
- Брандмауер або VPN-клієнт може впливати на перехоплення; для перевірки їх можна тимчасово вимкнути.

5. Приклад роботи програми

Нижче наведено приклад формату виведення.

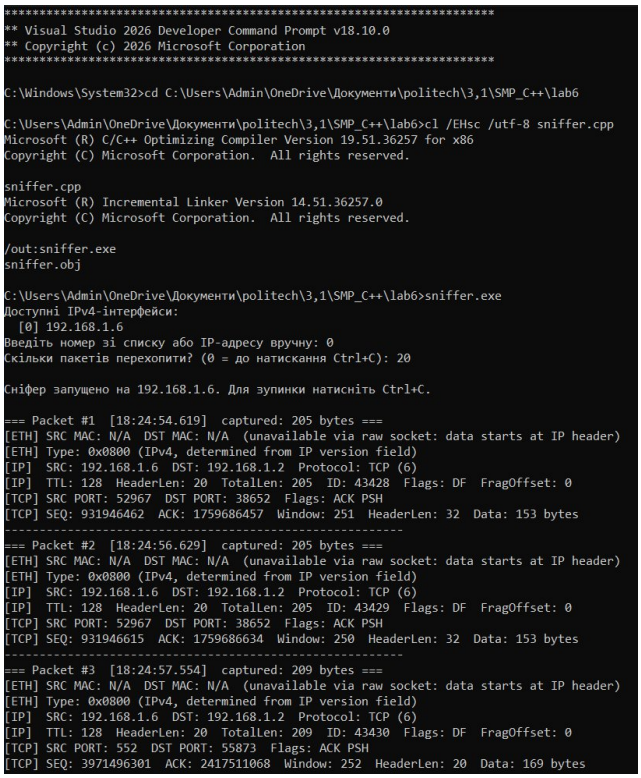


Рис. 1 — Збірка та запуск сніфера: вибір інтерфейсу 192.168.1.6 і перші перехоплені TCP-пакети

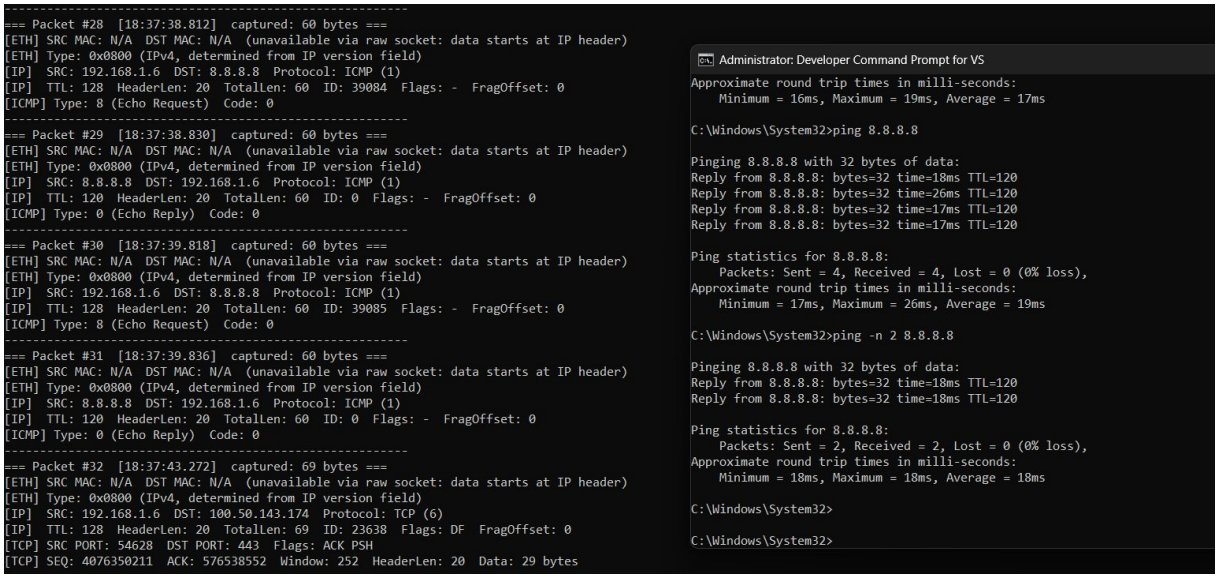


Рис. 2 — Перехоплення ICMP: Echo Request (Type 8) та Echo Reply (Type 0, TTL = 120) під час ping 8.8.8.8

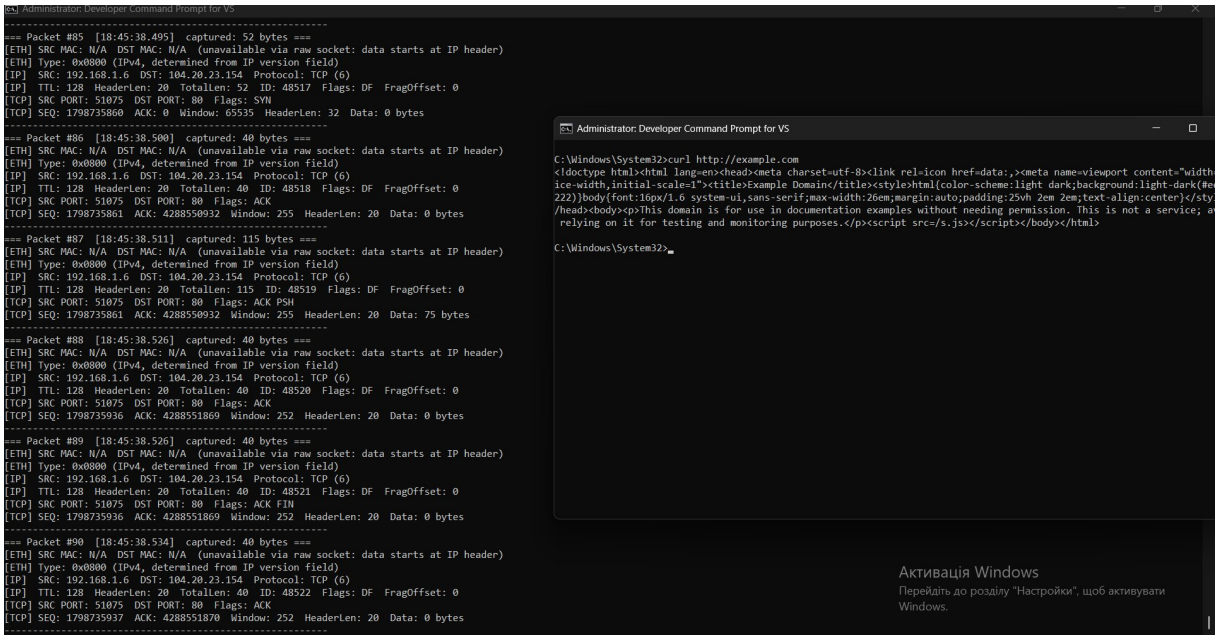


Рис. 3 — Перехоплення TCP під час curl http://example.com: SYN (порт 80), ACK, ACK PSH (HTTP-запит, 75 байт) і ACK FIN

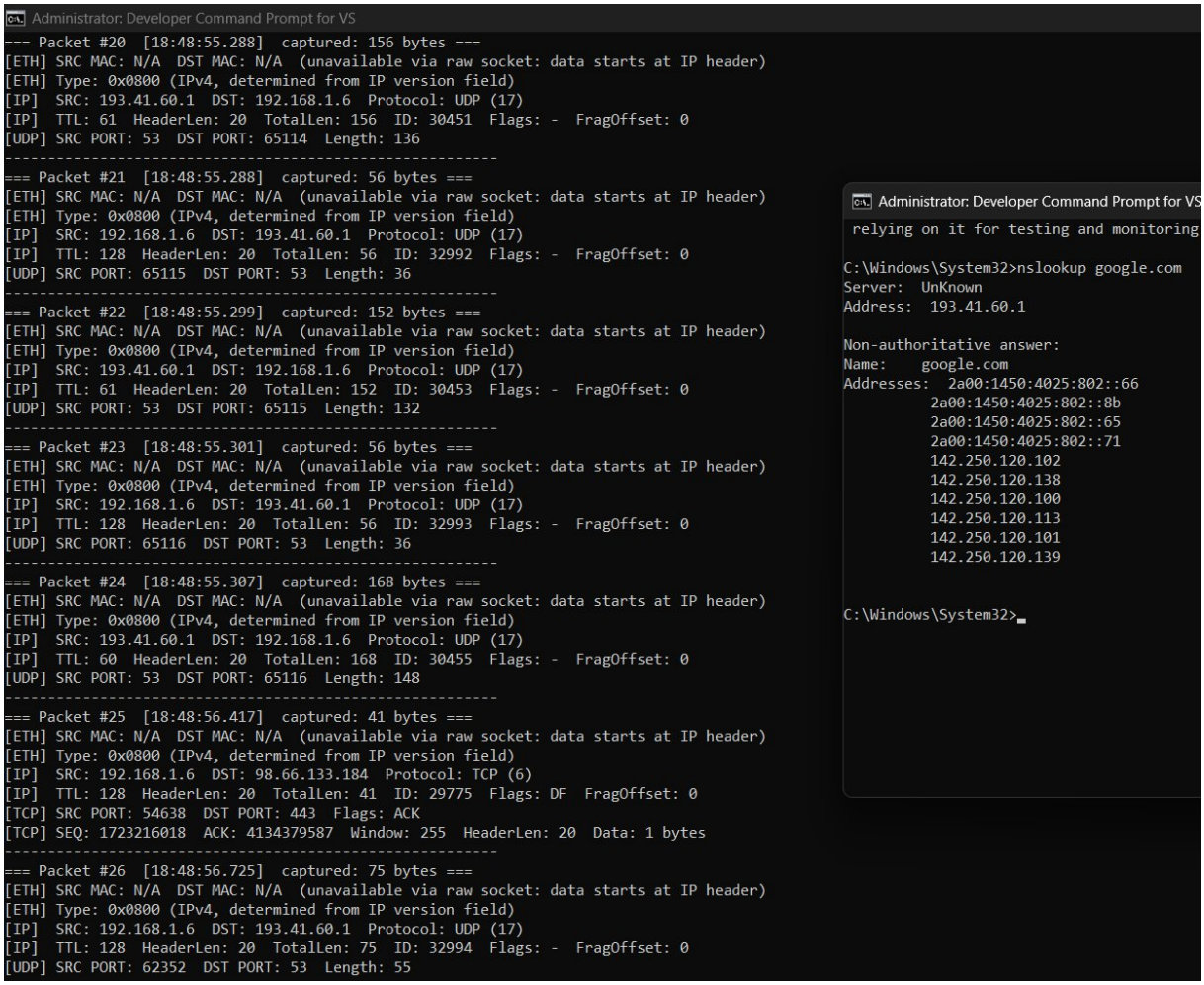


Рис. 4 — Перехоплення UDP/DNS під час nslookup google.com: запити та відповіді на порт 53 (DNS-сервер 193.41.60.1)

6. Висновки

У ході лабораторної роботи розроблено консольний мережевий сніфер мовою C++ з використанням WinAPI та Winsock2. Програма створює raw-сокет, прив'язує його до обраного інтерфейсу, вмикає режим прийому всіх пакетів через ioctlsocket з параметром SIO_RCVALL та в реальному часі розбирає заголовки IPv4, TCP, UDP і ICMP за допомогою власних структур із #pragma pack(1) та функцій ntohs/ntohl.

Під час роботи встановлено, що raw-сокет у Windows не повертає Ethernet-заголовок, тому MAC-адреси через Winsock отримати неможливо; цю частину виведено як «недоступно», а тип протоколу визначено за версією IP. Також засвоєно принципи розбору заголовків змінної довжини (IHL, Data Offset), виділення бітових полів (прапорці TCP, фрагментація IP) і перетворення порядку байтів між мережевим і хостовим.

Додаток А. Текст програми (sniffer.cpp)

```
// ============================================================================
// Лабораторна робота №6
// Консольний мережевий сніфер (C++, WinAPI + Winsock2, raw socket)
//
// Збірка (Developer Command Prompt for VS):
//     cl /EHsc /utf-8 sniffer.cpp
// Запуск: від імені адміністратора!
// ============================================================================

#define WIN32_LEAN_AND_MEAN
#define _CRT_SECURE_NO_WARNINGS
#include <winsock2.h>
#include <ws2tcpip.h>
#include <mstcpip.h>   // SIO_RCVALL, RCVALL_ON, RCVALL_OFF
#include <windows.h>

#include <cstdio>
#include <cstdlib>
#include <string>
#include <vector>
#include <iostream>

#pragma comment(lib, "ws2_32.lib")

// ----------------------------------------------------------------------------
// Структури заголовків (розбираються вручну). pack(1) - без вирівнювання,
// щоб розмір і зміщення полів збігались із форматом у мережі.
// ----------------------------------------------------------------------------
#pragma pack(push, 1)

struct eth_header {                 // 14 байт
    unsigned char  dst[6];
    unsigned char  src[6];
    unsigned short type;
};

struct ip_header {                  // 20 байт (без опцій)
    unsigned char  ihl_version;     // старші 4 біти - версія, молодші 4 - IHL
    unsigned char  tos;
    unsigned short total_length;
    unsigned short id;
    unsigned short flags_offset;    // 3 біти прапорців + 13 біт зміщення
    unsigned char  ttl;
    unsigned char  protocol;
    unsigned short checksum;
    unsigned int   src;
    unsigned int   dst;
};

struct tcp_header {                 // 20 байт (без опцій)
    unsigned short src_port;
    unsigned short dst_port;
    unsigned int   seq;
    unsigned int   ack;
    unsigned short offset_flags;    // 4 біти Data Offset, 3 Reserved, 9 прапорців
    unsigned short window;
    unsigned short checksum;
    unsigned short urgent_ptr;
};

struct udp_header {                 // 8 байт
    unsigned short src_port;
    unsigned short dst_port;
    unsigned short length;
    unsigned short checksum;
};

struct icmp_header {                // перші 4 байти ICMP
    unsigned char  type;
    unsigned char  code;
    unsigned short checksum;
```

```
};

#pragma pack(pop)

static_assert(sizeof(eth_header) == 14, "eth_header must be 14 bytes");
static_assert(sizeof(ip_header)  == 20, "ip_header must be 20 bytes");
static_assert(sizeof(tcp_header) == 20, "tcp_header must be 20 bytes");
static_assert(sizeof(udp_header) == 8,  "udp_header must be 8 bytes");

// ----------------------------------------------------------------------------
// Допоміжні змінні та функції
// ----------------------------------------------------------------------------
static volatile LONG g_running = 1;

// Обробник Ctrl+C: лише ставить прапорець, щоб цикл завершився коректно
static BOOL WINAPI ConsoleHandler(DWORD event)
{
    if (event == CTRL_C_EVENT || event == CTRL_BREAK_EVENT || event == CTRL_CLOSE_EVENT) {
        InterlockedExchange(&g_running, 0);
        return TRUE;
    }
    return FALSE;
}

static std::string IpToString(unsigned int ip_net_order)
{
    in_addr a;
    a.s_addr = ip_net_order;                // значення вже у network order
    char buf[INET_ADDRSTRLEN] = {};
    inet_ntop(AF_INET, &a, buf, sizeof(buf));
    return buf;
}

static const char* ProtoName(unsigned char p)
{
    switch (p) {
    case 1:  return "ICMP";
    case 6:  return "TCP";
    case 17: return "UDP";
    default: return "OTHER";
    }
}

static std::string TcpFlagsToString(unsigned short flags)
{
    static const struct { unsigned short mask; const char* name; } table[] = {
        { 0x002, "SYN" }, { 0x010, "ACK" }, { 0x001, "FIN" }, { 0x004, "RST" },
        { 0x008, "PSH" }, { 0x020, "URG" }, { 0x040, "ECE" }, { 0x080, "CWR" },
        { 0x100, "NS"  }
    };
    std::string s;
    for (const auto& t : table) {
        if (flags & t.mask) {
            if (!s.empty()) s += ' ';
            s += t.name;
        }
    }
    return s.empty() ? "-" : s;
}

static const char* IcmpTypeName(unsigned char t)
{
    switch (t) {
    case 0:  return "Echo Reply";
    case 3:  return "Destination Unreachable";
    case 5:  return "Redirect";
    case 8:  return "Echo Request";
    case 11: return "Time Exceeded";
    default: return "Other";
    }
}

// ----------------------------------------------------------------------------
// Розбір одного пакета
```

```
// ----------------------------------------------------------------------------
static void ProcessPacket(const unsigned char* buf, int size, unsigned long number)
{
    SYSTEMTIME st;
    GetLocalTime(&st);
    printf("=== Packet #%lu  [%02d:%02d:%02d.%03d]  captured: %d bytes ===\n",
           number, st.wHour, st.wMinute, st.wSecond, st.wMilliseconds, size);


    // --- Ethernet ---------------------------------------------------------
    // Raw socket у Windows (SIO_RCVALL) повертає пакет, що починається
    // одразу з IP-заголовка. Ethernet-заголовка (MAC-адрес) у даних НЕМАЄ,
    // тому отримати його через Winsock неможливо.
    printf("[ETH] SRC MAC: N/A  DST MAC: N/A  "
           "(unavailable via raw socket: data starts at IP header)\n");


    // --- IP ---------------------------------------------------------------
    if (size < (int)sizeof(ip_header)) {
        printf("[IP]  packet too short (%d bytes)\n", size);
        printf("-----------------------------------------------------------\n");
        return;
    }

    const ip_header* ip = (const ip_header*)buf;
    unsigned version   = ip->ihl_version >> 4;
    unsigned ihl       = ip->ihl_version & 0x0F;
    unsigned ip_hdrlen = ihl * 4;

    if (version != 4 || ip_hdrlen < 20 || (int)ip_hdrlen > size) {
        printf("[IP]  not a valid IPv4 packet (version=%u, IHL=%u)\n", version, ihl);
        printf("-----------------------------------------------------------\n");
        return;
    }

    printf("[ETH] Type: 0x0800 (IPv4, determined from IP version field)\n");

    unsigned short total_len = ntohs(ip->total_length);
    unsigned short frag      = ntohs(ip->flags_offset);
    unsigned frag_offset     = (frag & 0x1FFF) * 8;
    bool df = (frag & 0x4000) != 0;
    bool mf = (frag & 0x2000) != 0;

    printf("[IP]  SRC: %s  DST: %s  Protocol: %s (%u)\n",
           IpToString(ip->src).c_str(), IpToString(ip->dst).c_str(),
           ProtoName(ip->protocol), ip->protocol);
    printf("[IP]  TTL: %u  HeaderLen: %u  TotalLen: %u  ID: %u  Flags:%s%s  FragOffset: %u\n",
           ip->ttl, ip_hdrlen, total_len, ntohs(ip->id),
           df ? " DF" : "", mf ? " MF" : (df ? "" : " -"), frag_offset);

    // Межа даних IP-пакета: беремо менше з Total Length і реально отриманого
    int end = (total_len >= ip_hdrlen && (int)total_len <= size) ? (int)total_len : size;
    const unsigned char* payload = buf + ip_hdrlen;
    int payload_len = end - (int)ip_hdrlen;

    if (frag_offset != 0) {
        printf("[IP]  fragment (offset != 0): transport header is not present\n");
        printf("-----------------------------------------------------------\n");
        return;
    }

    // --- TCP / UDP / ICMP -------------------------------------------------
    switch (ip->protocol) {
    case 6: {   // TCP
        if (payload_len < (int)sizeof(tcp_header)) {
            printf("[TCP] truncated header\n");
            break;
        }
        const tcp_header* tcp = (const tcp_header*)payload;
        unsigned short of     = ntohs(tcp->offset_flags);
        unsigned data_offset  = (of >> 12) * 4;          // 4 старші біти, у 32-бітних словах
        unsigned short flags  = of & 0x01FF;             // 9 молодших бітів
        int data_len = payload_len - (int)data_offset;
        if (data_len < 0) data_len = 0;
```

```
        printf("[TCP] SRC PORT: %u  DST PORT: %u  Flags: %s\n",
               ntohs(tcp->src_port), ntohs(tcp->dst_port), TcpFlagsToString(flags).c_str());
        printf("[TCP] SEQ: %u  ACK: %u  Window: %u  HeaderLen: %u  Data: %d bytes\n",
               ntohl(tcp->seq), ntohl(tcp->ack), ntohs(tcp->window), data_offset, data_len);
        break;
    }
    case 17: {  // UDP
        if (payload_len < (int)sizeof(udp_header)) {
            printf("[UDP] truncated header\n");
            break;
        }
        const udp_header* udp = (const udp_header*)payload;
        printf("[UDP] SRC PORT: %u  DST PORT: %u  Length: %u\n",
               ntohs(udp->src_port), ntohs(udp->dst_port), ntohs(udp->length));
        break;
    }
    case 1: {   // ICMP
        if (payload_len < (int)sizeof(icmp_header)) {
            printf("[ICMP] truncated header\n");
            break;
        }
        const icmp_header* icmp = (const icmp_header*)payload;
        printf("[ICMP] Type: %u (%s)  Code: %u\n",
               icmp->type, IcmpTypeName(icmp->type), icmp->code);
        break;
    }
    default:
        printf("[%s] protocol %u is not parsed by this sniffer\n",
               ProtoName(ip->protocol), ip->protocol);
        break;
    }
    printf("------------------------------------------------------------\n");
}

// ----------------------------------------------------------------------------
// Отримання локальних IPv4-адрес (gethostname + getaddrinfo)
// ----------------------------------------------------------------------------
static std::vector<std::string> GetLocalIPv4()
{
    std::vector<std::string> result;
    char host[256] = {};
    if (gethostname(host, sizeof(host)) == SOCKET_ERROR)
        return result;

    addrinfo hints = {};
    hints.ai_family   = AF_INET;
    hints.ai_socktype = SOCK_STREAM;   // щоб не отримувати дублікати для різних типів сокетів
    hints.ai_protocol = IPPROTO_TCP;

    addrinfo* list = nullptr;
    if (getaddrinfo(host, nullptr, &hints, &list) != 0)
        return result;

    for (addrinfo* p = list; p != nullptr; p = p->ai_next) {
        const sockaddr_in* sa = (const sockaddr_in*)p->ai_addr;
        result.push_back(IpToString(sa->sin_addr.s_addr));
    }
    freeaddrinfo(list);
    return result;
}

// ----------------------------------------------------------------------------
// main
// ----------------------------------------------------------------------------
int main()
{
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCtrlHandler(ConsoleHandler, TRUE);

    // Крок 1. Ініціалізація Winsock
    WSADATA wsa;
    int rc = WSAStartup(MAKEWORD(2, 2), &wsa);
    if (rc != 0) {
        printf("WSAStartup failed: %d\n", rc);
        return 1;
```

```
    }

    // Крок 2. Вибір локального інтерфейсу
    std::vector<std::string> ips = GetLocalIPv4();
    printf("Доступні IPv4-інтерфейси:\n");
    for (size_t i = 0; i < ips.size(); ++i)
        printf("  [%zu] %s\n", i, ips[i].c_str());
    if (ips.empty())
        printf("  (не вдалося визначити автоматично)\n");

    printf("Введіть номер зі списку або IP-адресу вручну: ");
    std::string input;
    std::getline(std::cin, input);

    std::string local_ip;
    if (!input.empty() && input.find('.') == std::string::npos) {
        size_t idx = (size_t)atoi(input.c_str());
        if (idx < ips.size()) local_ip = ips[idx];
    } else {
        local_ip = input;
    }
    if (local_ip.empty()) {
        printf("Невірний вибір інтерфейсу.\n");
        WSACleanup();
        return 1;
    }

    printf("Скільки пакетів перехопити? (0 = до натискання Ctrl+C): ");
    std::getline(std::cin, input);
    unsigned long max_packets = (unsigned long)atol(input.c_str());

    // Крок 3. Створення сирого сокета
    SOCKET sock = socket(AF_INET, SOCK_RAW, IPPROTO_IP);
    if (sock == INVALID_SOCKET) {
        int err = WSAGetLastError();
        printf("socket() failed: %d\n", err);
        if (err == WSAEACCES)
            printf("Запустіть програму від імені адміністратора.\n");
        WSACleanup();
        return 1;
    }

    // Крок 4. Прив'язка до локального інтерфейсу
    sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port   = 0;
    if (inet_pton(AF_INET, local_ip.c_str(), &addr.sin_addr) != 1) {
        printf("Невірна IP-адреса: %s\n", local_ip.c_str());
        closesocket(sock);
        WSACleanup();
        return 1;
    }
    if (bind(sock, (sockaddr*)&addr, sizeof(addr)) == SOCKET_ERROR) {
        printf("bind() failed: %d\n", WSAGetLastError());
        closesocket(sock);
        WSACleanup();
        return 1;
    }

    // Таймаут recv = 1 с, щоб цикл міг перевіряти прапорець зупинки (Ctrl+C)
    DWORD timeout_ms = 1000;
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (const char*)&timeout_ms, sizeof(timeout_ms));

    // Крок 5. Режим прийому всіх пакетів (promiscuous)
    u_long flag = RCVALL_ON;   // = 1
    if (ioctlsocket(sock, SIO_RCVALL, &flag) == SOCKET_ERROR) {
        int err = WSAGetLastError();
        printf("ioctlsocket(SIO_RCVALL) failed: %d\n", err);
        if (err == WSAEACCES)
            printf("Запустіть програму від імені адміністратора.\n");
        closesocket(sock);
        WSACleanup();
        return 1;
    }
```

```
    printf("\nСніфер запущено на %s. Для зупинки натисніть Ctrl+C.\n\n", local_ip.c_str());

    // Крок 6-7. Цикл прийому та виведення
    static unsigned char buffer[65536];
    unsigned long count = 0;

    while (g_running && (max_packets == 0 || count < max_packets)) {
        int size = recv(sock, (char*)buffer, sizeof(buffer), 0);
        if (size == SOCKET_ERROR) {
            int err = WSAGetLastError();
            if (err == WSAETIMEDOUT)
                continue;                    // просто немає пакетів - чекаємо далі
            printf("recv() failed: %d\n", err);
            break;
        }
        if (size == 0)
            continue;

        ++count;
        ProcessPacket(buffer, size, count);
    }

    // Крок 8. Завершення роботи
    flag = RCVALL_OFF;
    ioctlsocket(sock, SIO_RCVALL, &flag);
    closesocket(sock);
    WSACleanup();

    printf("\nПерехоплено пакетів: %lu. Роботу завершено.\n", count);
    return 0;
}
```